

一、数据读取

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: # 从Excel中读取数据
dataset = pd.read_csv("../data/Breast_Cancer.csv")
```

```
In [3]: dataset.head()
```

	Age	Race	Marital Status	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Tumor Size
0	68	White	Married	T1	N1	IIA	Poorly differentiated	3	Regional	
1	50	White	Married	T2	N2	IIIA	Moderately differentiated	2	Regional	
2	58	White	Divorced	T3	N3	IIIC	Moderately differentiated	2	Regional	
3	58	White	Married	T1	N1	IIA	Poorly differentiated	3	Regional	
4	47	White	Married	T2	N1	IIB	Poorly differentiated	3	Regional	

该乳腺癌患者数据集是从 NCI 的 SEER 计划 2017 年 11 月更新中获得的，该计划提供了有关基于人群的癌症统计数据的信息。该数据集涉及 2006-2010 年诊断的浸润性导管癌和小叶癌乳腺癌（SEER 主要引用重码 NOS 组织学代码 8522/3）的女性患者。排除肿瘤大小未知、检查区域LN的患者、区域LN阳性、生存月数小于1个月的患者;因此，最终纳入了 4024 名患者。

特征描述

- Age | 年龄
- Race | 种族：白人，黑人，其他（美洲印第安人/阿拉斯加土著、亚洲/太平洋岛民
- Marital Status | 婚姻状况：已婚，单身，离婚，分居，丧偶
- T Stage | T分期：肿瘤直径为依据
- N Stage | N分期：局部淋巴结转移情况为依据
- 6th Stage | AJCC第六版分期：原位癌、肿瘤大小、局部淋巴结转移和器官转移等为依据
- differentiate | 分化程度：未分化，低分化，中等分化，高分化
- Grade | 等级：高等级意味着癌症增长更快，更容易扩散
- A Stage | A分期：局部（未转移），远处（转移）
- Tumor Size | 肿瘤大小
- Estrogen Status | 雌激素状态：ER阳性乳腺癌
- Progesterone Status | 孕激素状态：PR阳性乳腺癌

- Regional Node Examined | 检查的区域节点：记录病理检查的局部淋巴结总数
- Reginol Node Positive | 区域节点阳性数：记录检查局部阳性淋巴结的数量
- Survival Months | 生存时间（月）
- Status | 结局状态：存活，死亡

二、查看数据详细信息

In [4]: `dataset.info()` # 数据整体信息

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4024 entries, 0 to 4023
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   4024 non-null   int64
1   Race                                 4024 non-null   object
2   Marital Status                       4024 non-null   object
3   T Stage                              4024 non-null   object
4   N Stage                              4024 non-null   object
5   6th Stage                            4024 non-null   object
6   differentiate                         4024 non-null   object
7   Grade                                4024 non-null   object
8   A Stage                              4024 non-null   object
9   Tumor Size                           4024 non-null   int64
10  Estrogen Status                      4024 non-null   object
11  Progesterone Status                  4024 non-null   object
12  Regional Node Examined               4024 non-null   int64
13  Reginol Node Positive                 4024 non-null   int64
14  Survival Months                      4024 non-null   int64
15  Status                               4024 non-null   object
dtypes: int64(5), object(11)
memory usage: 503.1+ KB
```

In [5]: `dataset.nunique()` # 查看各列的不同值的数量

```
Out[5]: Age                40
Race                3
Marital Status      5
T Stage             4
N Stage             3
6th Stage           5
differentiate        4
Grade               4
A Stage             2
Tumor Size          110
Estrogen Status      2
Progesterone Status  2
Regional Node Examined  54
Reginol Node Positive  38
Survival Months      107
Status              2
dtype: int64
```

In [6]: `dataset['Race'].value_counts()` # 查看列的取值

```
Out[6]: Race
White    3413
Other     320
Black     291
Name: count, dtype: int64
```

```
In [7]: dataset.describe() # 定量资料的统计描述
```

```
Out[7]:
```

	Age	Tumor Size	Regional Node Examined	Reginol Node Positive	Survival Months
count	4024.000000	4024.000000	4024.000000	4024.000000	4024.000000
mean	53.972167	30.473658	14.357107	4.158052	71.297962
std	8.963134	21.119696	8.099675	5.109331	22.921430
min	30.000000	1.000000	1.000000	1.000000	1.000000
25%	47.000000	16.000000	9.000000	1.000000	56.000000
50%	54.000000	25.000000	14.000000	2.000000	73.000000
75%	61.000000	38.000000	19.000000	5.000000	90.000000
max	69.000000	140.000000	61.000000	46.000000	107.000000

三、数据可视化探索

```
In [8]: import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
%config InlineBackend.figure_format = 'retina'

plt.rcParams['font.sans-serif'] = ['FangSong'] # 指定字体，比如FangSong(仿宋)
plt.rcParams['axes.unicode_minus'] = False # 正常显示符号（-）
```

绘图基础

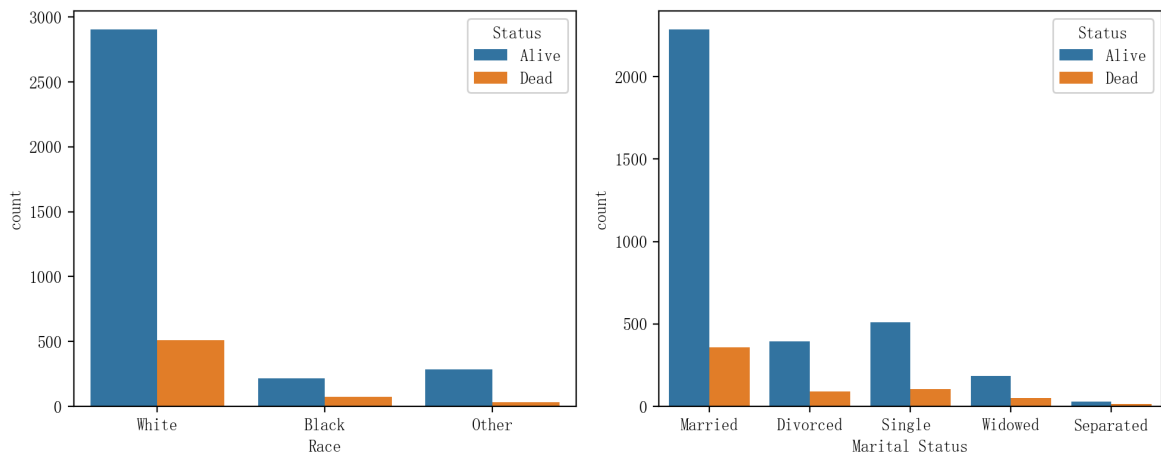
- 画布，坐标系: `fig, ax = plt.subplots(figsize=(m,n))`
- 直条图: `sns.countplot(data, x, y, hue, ax)`
- 直方图: `sns.histplot(data, x, y, bins, kde, hue, ax)`
- 散点图: `sns.scatterplot(data, x, y, hue, ax)`
- 箱线图: `sns.boxplot(data, x, y, ax)`
- 热图: `sns.heatmap(data, cmap, annot, ax)`
- 自定义图标题: `ax.set_title(label, fontsize)`
- 自定义横轴名: `ax.set_xlabel(label, fontsize)`
- 自定义纵轴名: `ax.set_ylabel(label, fontsize)`
- 保存图片: `plt.savefig(filename, dpi, bbox_inches)`

（一）定性数据可视化探索

```
In [9]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]    # 坐标系1
ax2 = ax[1]    # 坐标系2

sns.countplot(data=dataset, hue='Status', ax=ax1, x='Race',    )
sns.countplot(data=dataset, hue='Status', ax=ax2, x='Marital Status',)

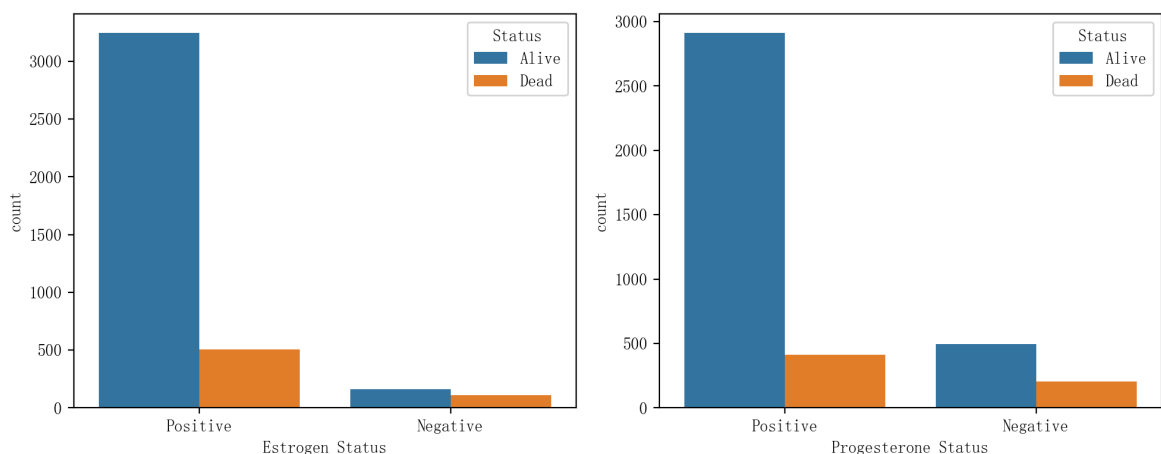
plt.tight_layout() # 自适应排版
```



```
In [10]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]    # 坐标系1
ax2 = ax[1]    # 坐标系2

sns.countplot(data=dataset, hue='Status', ax=ax1, x='Estrogen Status',    )
sns.countplot(data=dataset, hue='Status', ax=ax2, x='Progesterone Status',)

plt.tight_layout() # 自适应排版
```

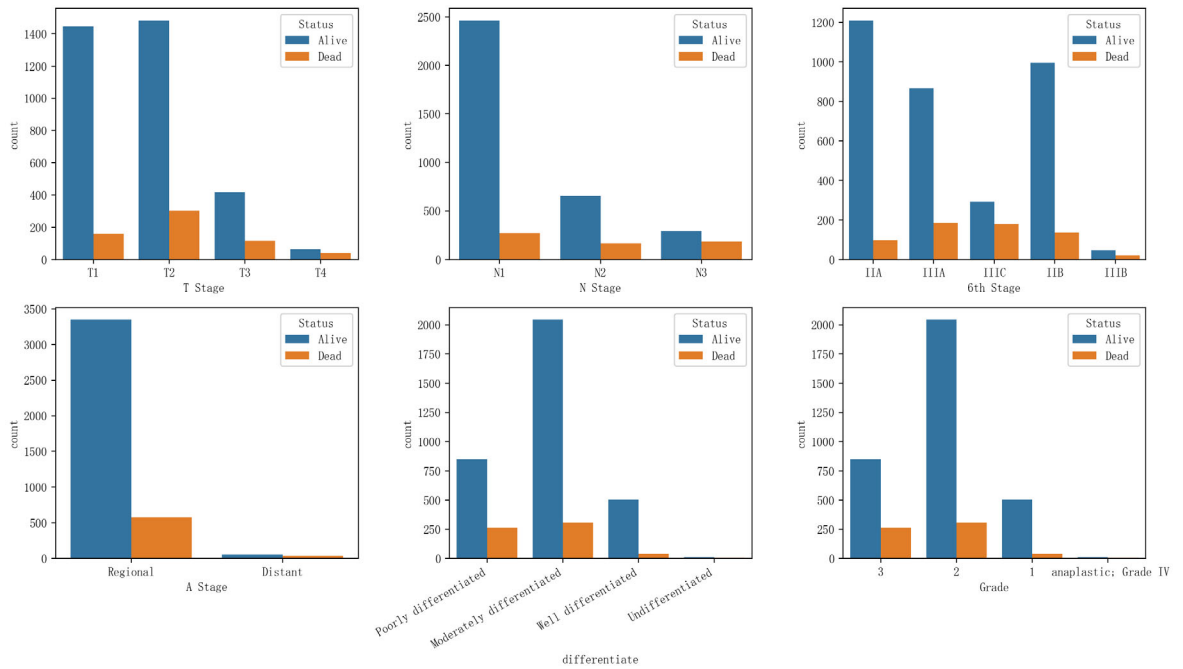


```
In [11]: fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(14,8))
ax1 = axes[0,0]
ax2 = axes[0,1]
ax3 = axes[0,2]
ax4 = axes[1,0]
ax5 = axes[1,1]
ax6 = axes[1,2]

sns.countplot(data=dataset, hue='Status', ax=ax1, x='T Stage ',    )
sns.countplot(data=dataset, hue='Status', ax=ax2, x='N Stage',    )
sns.countplot(data=dataset, hue='Status', ax=ax3, x='6th Stage',    )
sns.countplot(data=dataset, hue='Status', ax=ax4, x='A Stage',    )
```

```
sns.countplot(data=dataset, hue='Status', ax=ax5, x='differentiate',)
sns.countplot(data=dataset, hue='Status', ax=ax6, x='Grade',)

plt.setp(ax5.get_xticklabels(), rotation=30, ha="right") # 旋转刻度值标签
plt.tight_layout()
```

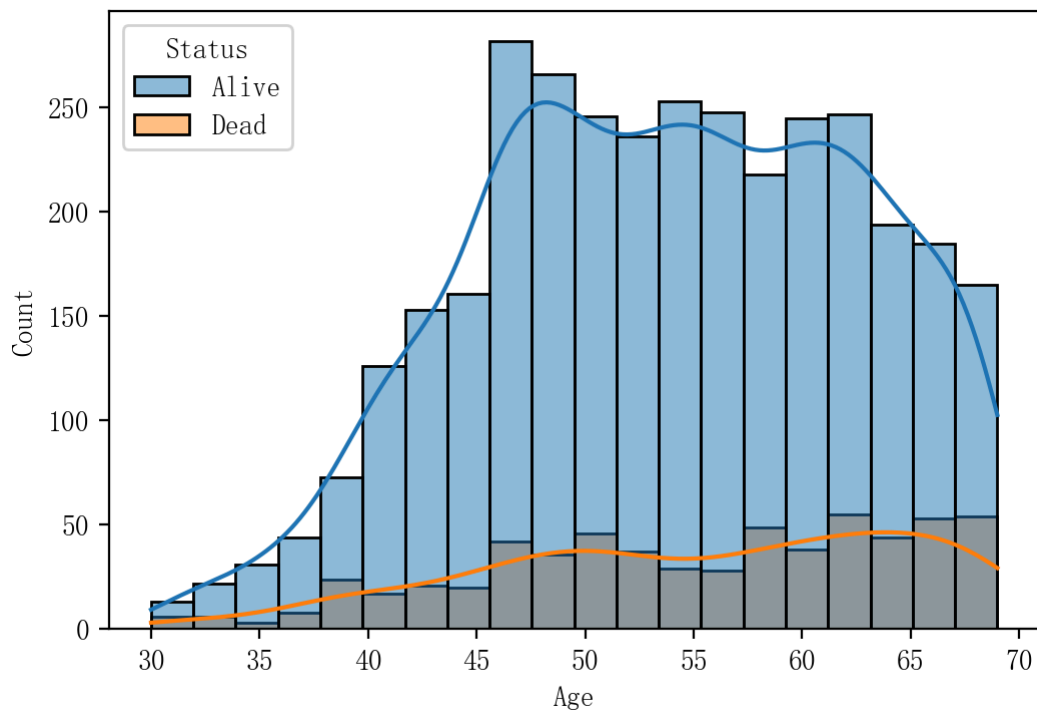


(二) 定量数据可视化探索

```
In [12]: # 直方图
fig, ax = plt.subplots(figsize=(6,4))

sns.histplot(data=dataset, bins=20, kde=True, ax=ax, hue='Status', x='Age')
```

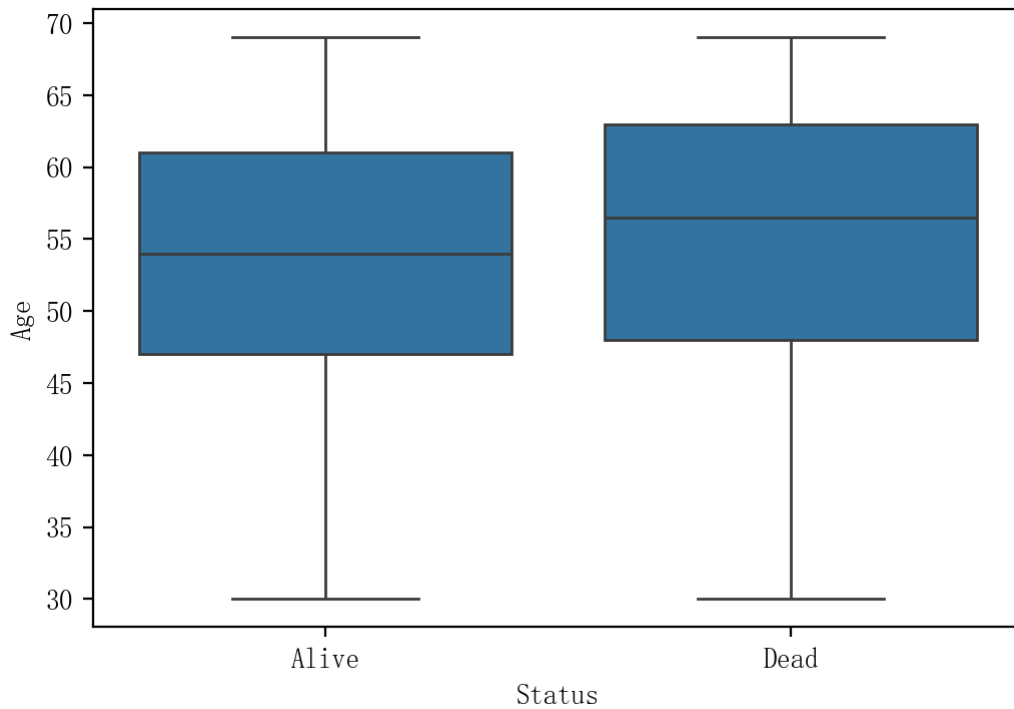
Out[12]: <Axes: xlabel='Age', ylabel='Count'>



```
In [13]: # 箱线图
fig, ax = plt.subplots(figsize=(6,4))

sns.boxplot(data=dataset, ax=ax, x='Status', y='Age')
```

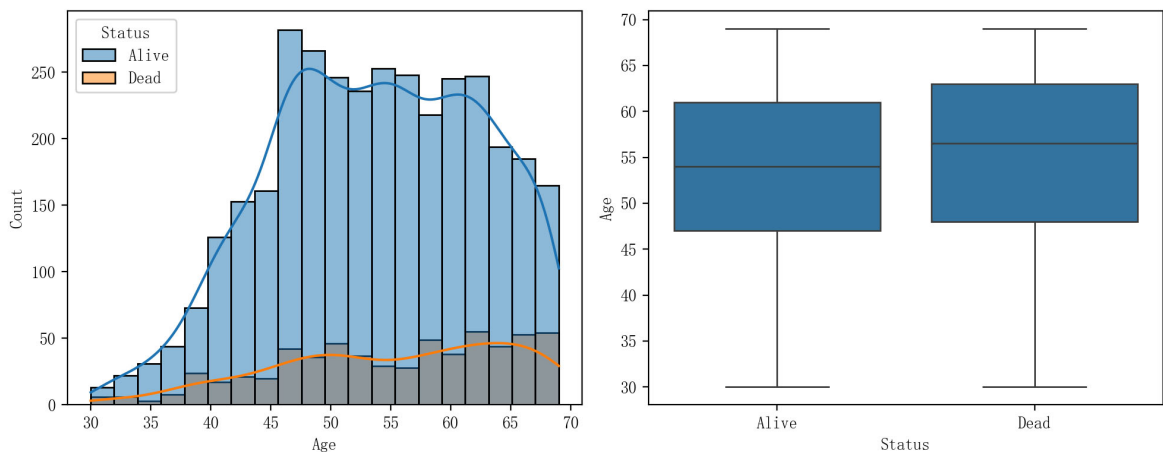
Out[13]: <Axes: xlabel='Status', ylabel='Age'>



```
In [14]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]
ax2 = ax[1]

sns.histplot(data=dataset, bins=20, kde=True, ax=ax1, hue='Status', x='Age')
sns.boxplot(data=dataset, ax=ax2, x='Status', y='Age')

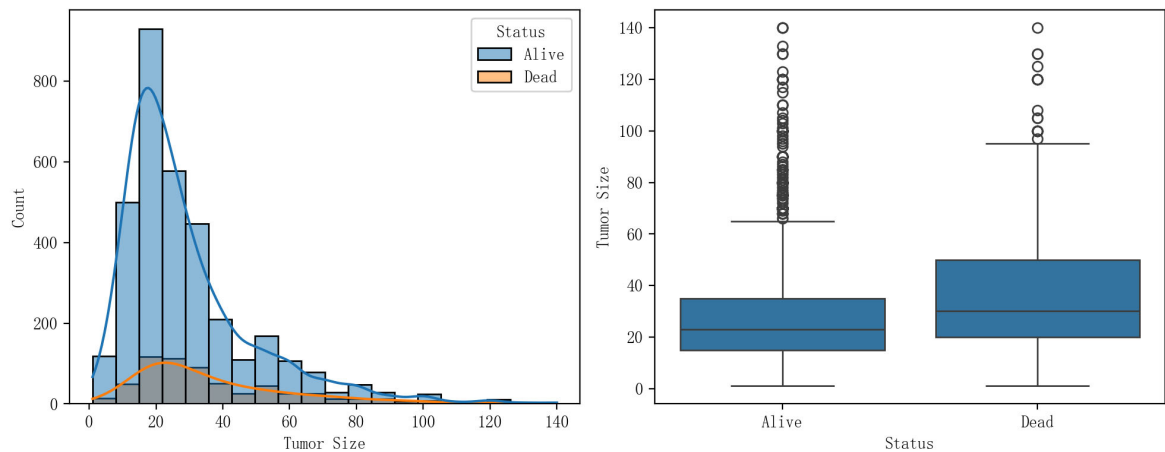
plt.tight_layout()
```



```
In [15]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]
ax2 = ax[1]

sns.histplot(data=dataset, bins=20, kde=True, ax=ax1, hue='Status', x='Tumor Siz
sns.boxplot(data=dataset, ax=ax2, x='Status', y='Tumor Size')
```

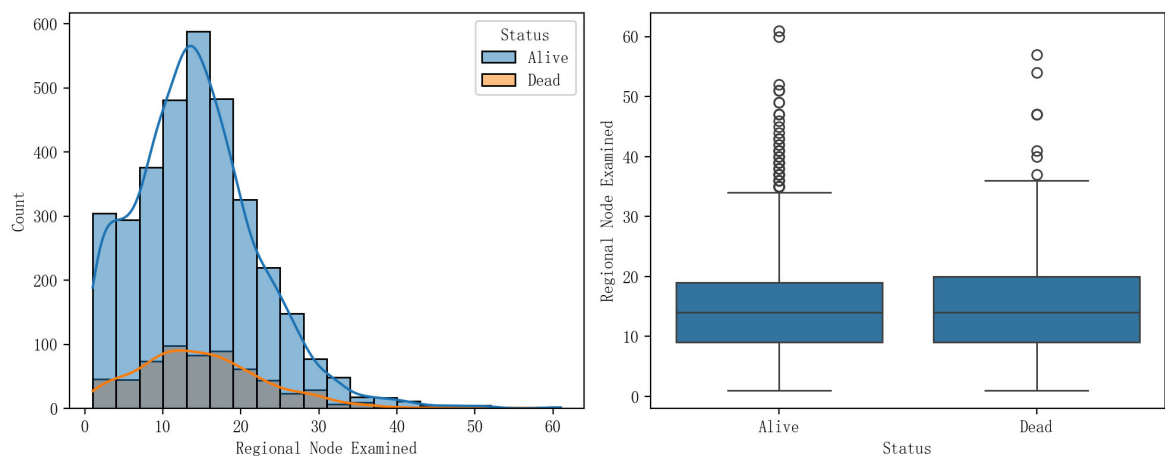
```
plt.tight_layout()
```



```
In [16]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]
ax2 = ax[1]

sns.histplot(data=dataset, bins=20, kde=True, ax=ax1, hue='Status', x='Regional Node Examined')
sns.boxplot(data=dataset, ax=ax2, x='Status', y='Regional Node Examined')

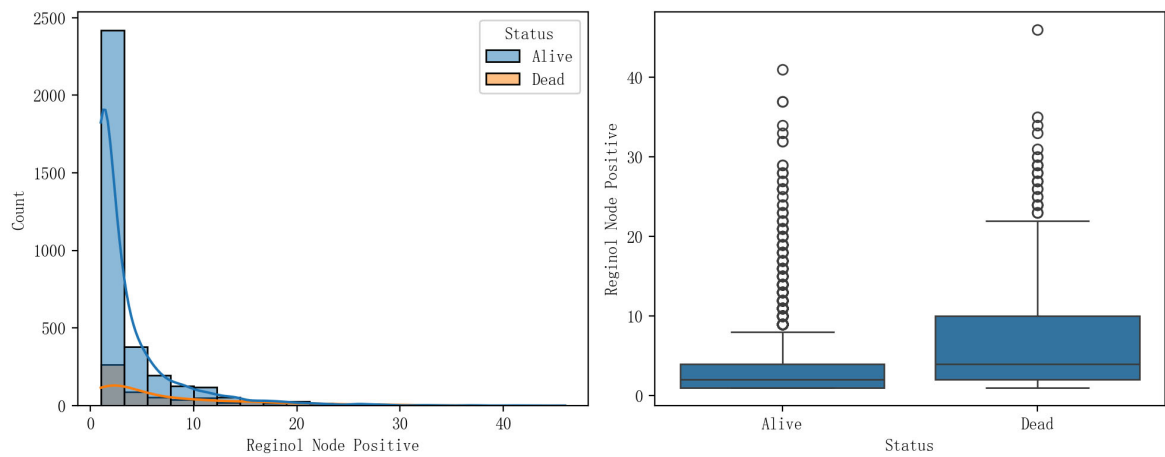
plt.tight_layout()
```



```
In [17]: fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,4))
ax1 = ax[0]
ax2 = ax[1]

sns.histplot(data=dataset, bins=20, kde=True, ax=ax1, hue='Status', x='Regional Node Examined')
sns.boxplot(data=dataset, ax=ax2, x='Status', y='Regional Node Examined')

plt.tight_layout()
```



四、数据预处理

```
In [18]: df = dataset.copy()
```

```
In [19]: df.head()
```

```
Out[19]:
```

	Age	Race	Marital Status	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Tumor Size
--	-----	------	----------------	---------	---------	-----------	---------------	-------	---------	------------

0	68	White	Married	T1	N1	IIA	Poorly differentiated	3	Regional	
1	50	White	Married	T2	N2	IIIA	Moderately differentiated	2	Regional	
2	58	White	Divorced	T3	N3	IIIC	Moderately differentiated	2	Regional	
3	58	White	Married	T1	N1	IIA	Poorly differentiated	3	Regional	
4	47	White	Married	T2	N1	IIB	Poorly differentiated	3	Regional	



```
In [20]: df.columns
```

```
Out[20]: Index(['Age', 'Race', 'Marital Status', 'T Stage ', 'N Stage', '6th Stage',
               'differentiate', 'Grade', 'A Stage', 'Tumor Size', 'Estrogen Status',
               'Progesterone Status', 'Regional Node Examined',
               'Reginol Node Positive', 'Survival Months', 'Status'],
              dtype='object')
```

```
In [21]: # 列名（特征名）重命名
df.columns = ['Age', 'Race', 'Marital Status', 'T Stage', 'N Stage', '6th Stage',
              'differentiate', 'Grade', 'A Stage', 'Tumor Size', 'Estrogen Status',
              'Progesterone Status', 'Regional Node Examined',
              'Regional Node Positive', 'Survival Months', 'Status']
```

```
In [22]: df.columns
```

```
Out[22]: Index(['Age', 'Race', 'Marital Status', 'T Stage', 'N Stage', '6th Stage',
              'differentiate', 'Grade', 'A Stage', 'Tumor Size', 'Estrogen Status',
              'Progesterone Status', 'Regional Node Examined',
              'Regional Node Positive', 'Survival Months', 'Status'],
              dtype='object')
```

```
In [23]: df['Grade'].value_counts()
```

```
Out[23]: Grade
2          2351
3          1111
1           543
anaplastic; Grade IV    19
Name: count, dtype: int64
```

```
In [24]: # 变量值重编码 df.loc[index, columns_name] = value

df.loc[df['Grade']=='1', 'Grade'] = 'I '
df.loc[df['Grade']=='2', 'Grade'] = 'II '
df.loc[df['Grade']=='3', 'Grade'] = 'III '
df.loc[df['Grade']=='anaplastic; Grade IV', 'Grade'] = 'IV'
```

```
In [25]: df['Grade'].value_counts()
```

```
Out[25]: Grade
II      2351
III     1111
I        543
IV        19
Name: count, dtype: int64
```

```
In [26]: df['Marital Status'].value_counts()
```

```
Out[26]: Marital Status
Married      2643
Single        615
Divorced      486
Widowed       235
Separated      45
Name: count, dtype: int64
```

```
In [27]: df.loc[df['Marital Status'] == 'Divorced', 'Marital Status'] = 'Other'
df.loc[df['Marital Status'] == 'Widowed', 'Marital Status'] = 'Other'
df.loc[df['Marital Status'] == 'Separated', 'Marital Status'] = 'Other'
```

```
In [28]: df['Marital Status'].value_counts()
```

```
Out[28]: Marital Status
Married      2643
Other         766
Single        615
Name: count, dtype: int64
```

```
In [29]: def age_to_group(age):
          if age >= 50:
              return 'elderly'
          elif age >= 40:
              return 'middle aged'
          elif age >= 30:
```

```

        return 'thirties'
    else:
        return 'young'

df['Age'] = df['Age'].apply(age_to_group)

```

In [30]: `df['Age'].value_counts()`

Out[30]:

```

Age
elderly      2670
middle aged  1124
thirties      230
Name: count, dtype: int64

```

In [31]: `df.head()`

Out[31]:

	Age	Race	Marital Status	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Tu
0	elderly	White	Married	T1	N1	IIA	Poorly differentiated	III	Regional	
1	elderly	White	Married	T2	N2	IIIA	Moderately differentiated	II	Regional	
2	elderly	White	Other	T3	N3	IIIC	Moderately differentiated	II	Regional	
3	elderly	White	Married	T1	N1	IIA	Poorly differentiated	III	Regional	
4	middle aged	White	Married	T2	N1	IIB	Poorly differentiated	III	Regional	

In [32]: `# 数据逻辑判断`
`df.loc[df['Regional Node Positive']>df['Regional Node Examined'],:]`

Out[32]:

	Age	Race	Marital Status	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Tumor Size	Es
--	-----	------	----------------	---------	---------	-----------	---------------	-------	---------	------------	----

五、统计检验

In [33]: `# 正态性检验 Shapiro-Wilk test`
`from scipy.stats import shapiro`
`num_columns = ['Tumor Size', 'Regional Node Examined', 'Regional Node Positive',`
`for column in num_columns:`
 `stat,p = shapiro(df[column])`
 `print(column, "\n 统计量: %f " %stat, " p值为: %f \n" %p)`

Tumor Size
统计量: 0.841386 p值为: 0.000000

Regional Node Examined
统计量: 0.959944 p值为: 0.000000

Regional Node Positive
统计量: 0.653169 p值为: 0.000000

Survival Months
统计量: 0.962397 p值为: 0.000000

```
In [34]: # 方差齐性检验 Levene检验
from scipy.stats import levene
num_columns = ['Tumor Size', 'Regional Node Examined', 'Regional Node Positive',
for column in num_columns:
    d1 = df.loc[df['Status']=='Alive', column]
    d2 = df.loc[df['Status']=='Dead', column]
    stat, p = levene(d1, d2)
    print(column, "\n 统计量: %f " %stat, " p值为: %f \n" %p)
```

Tumor Size
统计量: 25.620942 p值为: 0.000000

Regional Node Examined
统计量: 2.684287 p值为: 0.101421

Regional Node Positive
统计量: 218.336354 p值为: 0.000000

Survival Months
统计量: 48.542707 p值为: 0.000000

```
In [35]: # 独立样本t检验
from scipy.stats import ttest_ind
num_columns = ['Tumor Size', 'Regional Node Positive', 'Survival Months']
for column in num_columns:
    d1 = df.loc[df['Status']=='Alive', column]
    d2 = df.loc[df['Status']=='Dead', column]
    stat, p = ttest_ind(d1, d2)
    print(column, "\n 统计量: %f " %stat, " p值为: %f \n" %p)
```

Tumor Size
统计量: -8.588885 p值为: 0.000000

Regional Node Positive
统计量: -16.839798 p值为: 0.000000

Survival Months
统计量: 34.373652 p值为: 0.000000

```
In [36]: # Kruskal-wallis H检验
from scipy.stats import kruskal
num_columns = ['Tumor Size', 'Regional Node Examined', 'Regional Node Positive',
for column in num_columns:
    d1 = df.loc[df['Status']=='Alive', column]
    d2 = df.loc[df['Status']=='Dead', column]
```

```
stat,p = kruskal(d1,d2)
print(column, "\n 统计量: %f " %stat, " p值为: %f \n" %p)
```

Tumor Size

统计量: 79.489347 p值为: 0.000000

Regional Node Examined

统计量: 3.347812 p值为: 0.067294

Regional Node Positive

统计量: 193.064051 p值为: 0.000000

Survival Months

统计量: 694.131827 p值为: 0.000000

```
In [37]: # 卡方检验
from scipy.stats import chi2_contingency
num_columns = ['Age', 'Race', 'Marital Status', 'T Stage', 'N Stage', '6th Stage',
               'differentiate', 'Grade', 'A Stage', 'Estrogen Status', 'Progesterone Status']
for column in num_columns:
    stat,p,dof,expected = chi2_contingency(pd.crosstab(df[column],df['Status']))
    print(column, "\n 统计量: %f " %stat, " p值: %f \n" %p)
```

Age

统计量: 15.289546 p值: 0.000479

Race

统计量: 27.970007 p值: 0.000001

Marital Status

统计量: 21.145650 p值: 0.000026

T Stage

统计量: 103.476309 p值: 0.000000

N Stage

统计量: 269.929143 p值: 0.000000

6th Stage

统计量: 281.648443 p值: 0.000000

differentiate

统计量: 112.556276 p值: 0.000000

Grade

统计量: 112.556276 p值: 0.000000

A Stage

统计量: 35.764727 p值: 0.000000

Estrogen Status

统计量: 135.155739 p值: 0.000000

Progesterone Status

统计量: 124.885388 p值: 0.000000

六、数据编码

(一) 定量资料

```
In [38]: X_num = df[['Tumor Size', 'Regional Node Examined', 'Regional Node Positive', 'Survival Months']]
X_num.head()
```

```
Out[38]:
```

	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months
0	4	21	1	60
1	35	14	5	62
2	63	14	7	75
3	18	2	1	84
4	41	3	1	50

```
In [39]: X_num.describe()
```

```
Out[39]:
```

	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months
count	4024.000000	4024.000000	4024.000000	4024.000000
mean	30.473658	14.357107	4.158052	71.297962
std	21.119696	8.099675	5.109331	22.921430
min	1.000000	1.000000	1.000000	1.000000
25%	16.000000	9.000000	1.000000	56.000000
50%	25.000000	14.000000	2.000000	73.000000
75%	38.000000	19.000000	5.000000	90.000000
max	140.000000	61.000000	46.000000	107.000000

```
In [40]: # 数据标准化
X_num_norm = (X_num - X_num.mean()) / X_num.std()
```

```
In [41]: X_num_norm.head()
```

Out[41]:	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months
0	-1.253506	1.190528	-0.618095	-0.492900
1	0.214319	-0.044089	0.164786	-0.405645
2	1.540095	-0.044089	0.556227	0.161510
3	-0.590617	-1.525630	-0.618095	0.554156
4	0.498414	-1.402168	-0.618095	-0.929173

In [42]: X_num_norm.describe()

Out[42]:	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months
count	4.024000e+03	4.024000e+03	4.024000e+03	4.024000e+03
mean	4.679270e-17	5.650439e-17	1.187475e-16	3.063598e-16
std	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00
min	-1.395553e+00	-1.649092e+00	-6.180949e-01	-3.066910e+00
25%	-6.853156e-01	-6.613978e-01	-6.180949e-01	-6.674087e-01
50%	-2.591731e-01	-4.408910e-02	-4.223746e-01	7.425531e-02
75%	3.563660e-01	5.732196e-01	1.647864e-01	8.159193e-01
max	5.185981e+00	5.758613e+00	8.189320e+00	1.557583e+00

(二)定性资料

二分类资料或等级资料

```
In [43]: X_deg = df[['T Stage', 'N Stage', '6th Stage', 'differentiate',
                    'Grade', 'A Stage', 'Estrogen Status', 'Progesterone Status', 'Status']]

X_deg.head()
```

Out[43]:

	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Estrogen Status	Progesterone Status	St
0	T1	N1	IIA	Poorly differentiated	III	Regional	Positive	Positive	
1	T2	N2	IIIA	Moderately differentiated	II	Regional	Positive	Positive	
2	T3	N3	IIIC	Moderately differentiated	II	Regional	Positive	Positive	
3	T1	N1	IIA	Poorly differentiated	III	Regional	Positive	Positive	
4	T2	N1	IIB	Poorly differentiated	III	Regional	Positive	Positive	

In [44]:

```
## 编码
## pd.set_option('mode.chained_assignment', None)
# X_deg.loc[:, 'Status'] = X_deg['Status'].map({'Alive':0, 'Dead':1}).astype("int")
# X_deg.loc[:, 'Grade'] = X_deg['Grade'].map({'I':0, 'II':1, 'III':2, 'IV':3}).as
# X_deg.loc[:, 'T Stage'] = X_deg['T Stage'].map({'T1':0, 'T2':1, 'T3':2, 'T4':3})
# X_deg.loc[:, 'N Stage'] = X_deg['N Stage'].map({'N1':0, 'N2':1, 'N3':2}).astype
# X_deg.loc[:, 'A Stage'] = X_deg['A Stage'].map({'Regional':0, 'Distant':1}).ast
# X_deg.loc[:, '6th Stage'] = X_deg['6th Stage'].map({'IIA':0, 'IIB':1, 'IIIA':2,
# X_deg.loc[:, 'Estrogen Status'] = X_deg['Estrogen Status'].map({'Negative':0, '
# X_deg.loc[:, 'Progesterone Status'] = X_deg['Progesterone Status'].map({'Negati
# X_deg.loc[:, 'differentiate'] = X_deg['differentiate'].map({'Well differentiate
```

In [45]:

```
def safe_encode(series, mapping):
    """
    安全地进行列编码，处理未映射的值
    """
    encoded = series.map(mapping)
    # 检查是否有未映射的值
    unmapped = series[encoded.isna() & series.notna()].unique()
    if len(unmapped) > 0:
        print(f"警告：列 {series.name} 有未映射的值：{unmapped}")
    return encoded

# 应用编码
mappings = {
    'Status': {'Alive': 0, 'Dead': 1},
    'Grade': {'I': 0, 'II': 1, 'III': 2, 'IV': 3},
    'T Stage': {'T1': 0, 'T2': 1, 'T3': 2, 'T4': 3},
    'N Stage': {'N1': 0, 'N2': 1, 'N3': 2},
    'A Stage': {'Regional': 0, 'Distant': 1},
    '6th Stage': {'IIA': 0, 'IIB': 1, 'IIIA': 2, 'IIIB': 3, 'IIIC': 4},
    'Estrogen Status': {'Negative': 0, 'Positive': 1},
    'Progesterone Status': {'Negative': 0, 'Positive': 1},
    'differentiate': {'Well differentiated': 0, 'Moderately differentiated': 1,
                      'Poorly differentiated': 2, 'Undifferentiated': 3}
}

for col, mapping in mappings.items():
    if col in X_deg.columns:
        X_deg.loc[:, col] = safe_encode(X_deg[col], mapping).astype("int64")
```

```
In [46]: X_deg.head()
```

```
Out[46]:
```

	T Stage	N Stage	6th Stage	differentiate	Grade	A Stage	Estrogen Status	Progesterone Status	Statu
0	0	0	0		2	2	0	1	
1	1	1	2		1	1	0	1	
2	2	2	4		1	1	0	1	
3	0	0	0		2	2	0	1	
4	1	0	1		2	2	0	1	



```
In [47]: X_deg.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4024 entries, 0 to 4023
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   T Stage                4024 non-null  object
1   N Stage                4024 non-null  object
2   6th Stage              4024 non-null  object
3   differentiate           4024 non-null  object
4   Grade                  4024 non-null  object
5   A Stage                4024 non-null  object
6   Estrogen Status        4024 non-null  object
7   Progesterone Status    4024 non-null  object
8   Status                 4024 non-null  object
dtypes: object(9)
memory usage: 283.1+ KB
```

无序多分类变量

```
In [48]: X_cat = df[['Age', 'Race', 'Marital Status']]
X_cat.head()
```

```
Out[48]:
```

	Age	Race	Marital Status
0	elderly	White	Married
1	elderly	White	Married
2	elderly	White	Other
3	elderly	White	Married
4	middle aged	White	Married

```
In [49]: X_cat = pd.get_dummies(X_cat).astype("int64")
```

```
In [50]: X_cat.head()
```

Out[50]:

	Age_elderly	Age_middle aged	Age_thirties	Race_Black	Race_Other	Race_White	Sta
0	1	0	0	0	0	1	
1	1	0	0	0	0	1	
2	1	0	0	0	0	1	
3	1	0	0	0	0	1	
4	0	1	0	0	0	1	

合并编码后的数据资料

In [51]: `data = pd.concat([X_num_norm.astype("float64"), X_cat.astype("int64"), X_deg.ast`
`data.head()`

Out[51]:

	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months	Age_elderly	Age_middle aged	Age_thirti
0	-1.253506	1.190528	-0.618095	-0.492900	1	0	
1	0.214319	-0.044089	0.164786	-0.405645	1	0	
2	1.540095	-0.044089	0.556227	0.161510	1	0	
3	-0.590617	-1.525630	-0.618095	0.554156	1	0	
4	0.498414	-1.402168	-0.618095	-0.929173	0	1	

5 rows × 22 columns

In [52]: `data.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4024 entries, 0 to 4023
Data columns (total 22 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   Tumor Size                            4024 non-null   float64
 1   Regional Node Examined                4024 non-null   float64
 2   Regional Node Positive                 4024 non-null   float64
 3   Survival Months                       4024 non-null   float64
 4   Age_elderly                           4024 non-null   int64  
 5   Age_middle aged                       4024 non-null   int64  
 6   Age_thirties                          4024 non-null   int64  
 7   Race_Black                            4024 non-null   int64  
 8   Race_Other                            4024 non-null   int64  
 9   Race_White                            4024 non-null   int64  
10  Marital Status_Married                4024 non-null   int64  
11  Marital Status_Other                  4024 non-null   int64  
12  Marital Status_Single                 4024 non-null   int64  
13  T Stage                               4024 non-null   int64  
14  N Stage                               4024 non-null   int64  
15  6th Stage                             4024 non-null   int64  
16  differentiate                         4024 non-null   int64  
17  Grade                                 4024 non-null   int64  
18  A Stage                               4024 non-null   int64  
19  Estrogen Status                       4024 non-null   int64  
20  Progesterone Status                   4024 non-null   int64  
21  Status                                4024 non-null   int64  
dtypes: float64(4), int64(18)
memory usage: 691.8 KB

```

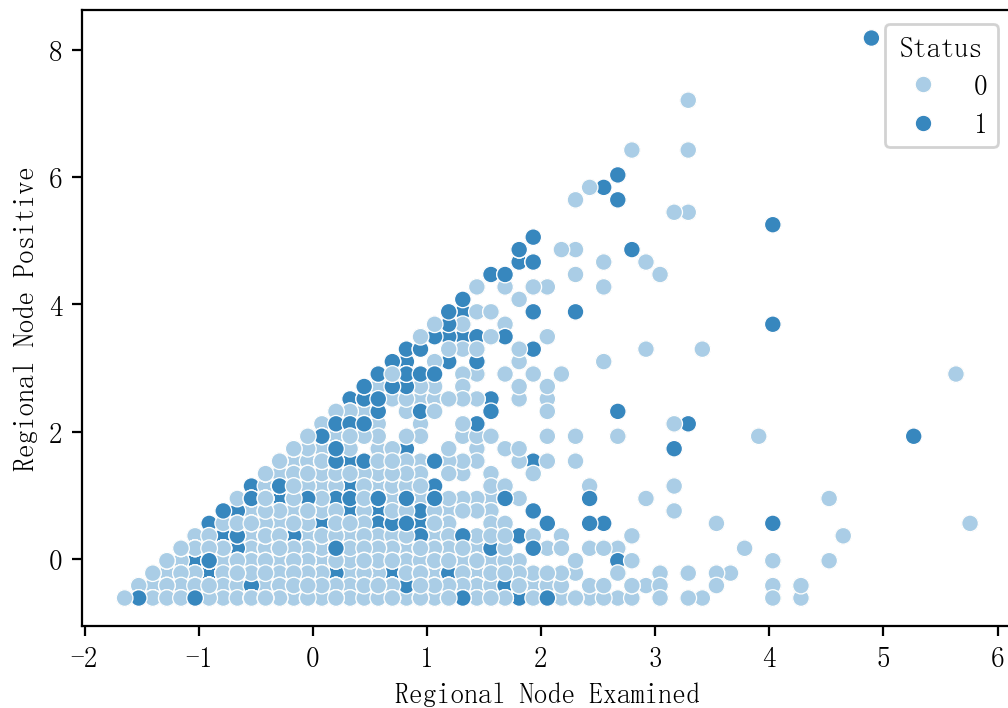
七、相关性分析

```

In [53]: # 散点图
fig, ax = plt.subplots(figsize=(6,4))
ax = sns.scatterplot(data=data, x='Regional Node Examined', y='Regional Node Pos
                    hue='Status', palette='Blues')

plt.show()

```



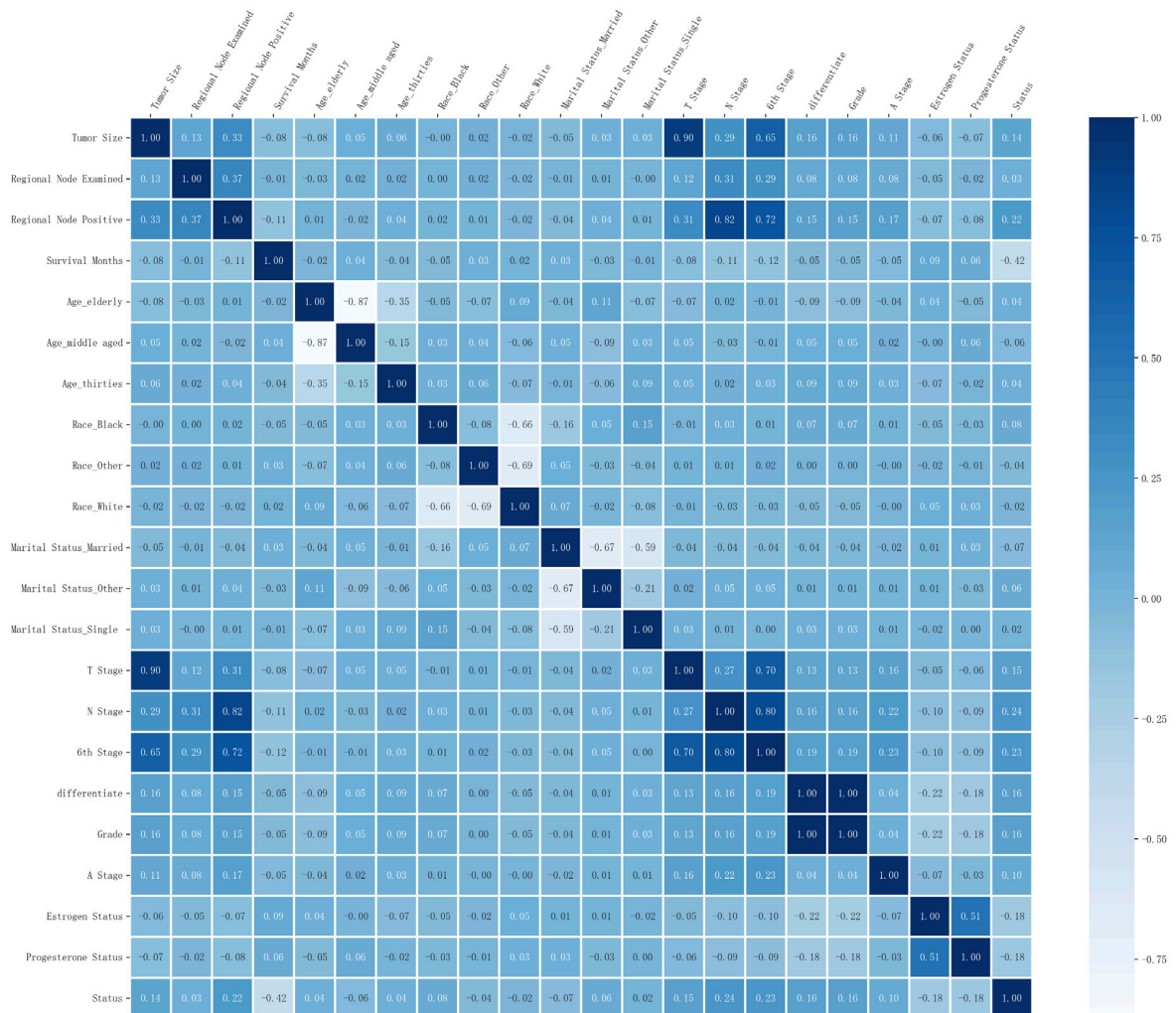
```
In [54]: # 相关系数矩阵
dcorr = data.corr(method='spearman')
dcorr.iloc[:5,:5]
```

Out[54]:

	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months	Age_elderly
Tumor Size	1.000000	0.128252	0.331297	-0.081247	-0.079699
Regional Node Examined	0.128252	1.000000	0.367189	-0.011049	-0.027968
Regional Node Positive	0.331297	0.367189	1.000000	-0.109792	0.005843
Survival Months	-0.081247	-0.011049	-0.109792	1.000000	-0.020679
Age_elderly	-0.079699	-0.027968	0.005843	-0.020679	1.000000

```
In [55]: # 相关系数热图
fig, ax = plt.subplots(figsize=(20,16))
sns.heatmap(dcorr, cmap='Blues', annot=True, fmt='.2f', linewidths=2, linecolor=

ax.tick_params(bottom=False, labelbottom=False, top=True, labeltop=True) #修改坐标轴
plt.setp(ax.get_xticklabels(), rotation=60, ha="left") # 旋转坐标轴标签的显示
plt.show()
```



八、逻辑回归分析

```
In [56]: X = data.iloc[:, :-1]
# 人工添加截距项
# X['intercept'] = 1.0
X.head()
```

```
Out[56]:
```

	Tumor Size	Regional Node Examined	Regional Node Positive	Survival Months	Age_elderly	Age_middle aged	Age_thirties
0	-1.253506	1.190528	-0.618095	-0.492900	1	0	
1	0.214319	-0.044089	0.164786	-0.405645	1	0	
2	1.540095	-0.044089	0.556227	0.161510	1	0	
3	-0.590617	-1.525630	-0.618095	0.554156	1	0	
4	0.498414	-1.402168	-0.618095	-0.929173	0	1	

5 rows × 7 columns



```
In [57]: # 划分训练数据和测试数据
x_train, y_train = X.iloc[:3001, :], data.iloc[:3001, -1]
```

```
x_test, y_test = X.iloc[3001:, :], data.iloc[3001:, -1]
```

```
In [58]: X.columns
```

```
Out[58]: Index(['Tumor Size', 'Regional Node Examined', 'Regional Node Positive',  
              'Survival Months', 'Age_elderly', 'Age_middle aged', 'Age_thirties',  
              'Race_Black', 'Race_Other', 'Race_White', 'Marital Status_Married',  
              'Marital Status_Other', 'Marital Status_Single ', 'T Stage', 'N Stage',  
              '6th Stage', 'differentiate', 'Grade', 'A Stage', 'Estrogen Status',  
              'Progesterone Status'],  
             dtype='object')
```

```
In [59]: cols = ['Regional Node Positive',  
                'Race_Other',  
                'Marital Status_Other', 'T Stage', 'N Stage',  
                'Grade', 'Estrogen Status',  
                'Progesterone Status']
```

```
In [60]: x_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3001 entries, 0 to 3000  
Data columns (total 21 columns):  
#   Column                                Non-Null Count  Dtype  
---  ---                                -  
0   Tumor Size                          3001 non-null   float64  
1   Regional Node Examined              3001 non-null   float64  
2   Regional Node Positive              3001 non-null   float64  
3   Survival Months                    3001 non-null   float64  
4   Age_elderly                        3001 non-null   int64  
5   Age_middle aged                    3001 non-null   int64  
6   Age_thirties                       3001 non-null   int64  
7   Race_Black                         3001 non-null   int64  
8   Race_Other                         3001 non-null   int64  
9   Race_White                        3001 non-null   int64  
10  Marital Status_Married              3001 non-null   int64  
11  Marital Status_Other               3001 non-null   int64  
12  Marital Status_Single              3001 non-null   int64  
13  T Stage                           3001 non-null   int64  
14  N Stage                           3001 non-null   int64  
15  6th Stage                         3001 non-null   int64  
16  differentiate                      3001 non-null   int64  
17  Grade                             3001 non-null   int64  
18  A Stage                           3001 non-null   int64  
19  Estrogen Status                   3001 non-null   int64  
20  Progesterone Status               3001 non-null   int64  
dtypes: float64(4), int64(17)  
memory usage: 492.5 KB
```

```
In [61]: import statsmodels.api as sm  
import statsmodels.formula.api as smf  
  
model = sm.Logit(y_train, x_train[cols]).fit()  
model.summary() # Logit分析结果
```

```
Optimization terminated successfully.  
Current function value: 0.385141  
Iterations 6
```

Out[61]:

Logit Regression Results

Dep. Variable:	Status	No. Observations:	3001
Model:	Logit	Df Residuals:	2993
Method:	MLE	Df Model:	7
Date:	Thu, 13 Nov 2025	Pseudo R-squ.:	0.1115
Time:	11:48:05	Log-Likelihood:	-1155.8
converged:	True	LL-Null:	-1300.8
Covariance Type:	nonrobust	LLR p-value:	8.410e-59

	coef	std err	z	P> z	[0.025	0.975]
Regional Node Positive	0.3559	0.076	4.686	0.000	0.207	0.505
Race_Other	-0.6772	0.234	-2.899	0.004	-1.135	-0.219
Marital Status_Other	0.2056	0.130	1.580	0.114	-0.049	0.461
T Stage	0.2365	0.067	3.547	0.000	0.106	0.367
N Stage	0.2560	0.120	2.139	0.032	0.021	0.491
Grade	-0.0507	0.069	-0.733	0.464	-0.186	0.085
Estrogen Status	-1.6161	0.156	-10.327	0.000	-1.923	-1.309
Progesterone Status	-0.6680	0.143	-4.676	0.000	-0.948	-0.388

In [62]:

```
# OR值 和 95%置信区间
import numpy as np

params = np.exp(model.params)
conf = np.exp(model.conf_int())
conf['OR'] = params
conf.columns = ['2.5%', '97.5%', 'OR']
conf
```

Out[62]:

	2.5%	97.5%	OR
Regional Node Positive	1.230021	1.656468	1.427407
Race_Other	0.321398	0.803021	0.508025
Marital Status_Other	0.951823	1.585009	1.228270
T Stage	1.111620	1.443706	1.266828
N Stage	1.021650	1.633248	1.291746
Grade	0.829933	1.088669	0.950538
Estrogen Status	0.146186	0.269977	0.198663
Progesterone Status	0.387506	0.678389	0.512718

In [63]:

```
fig, ax = plt.subplots(figsize=(6,5))

for d,c in zip(conf.index, conf.values):
    if d in ['intercept', 'Age_elderly', 'Marital Status_Single ']:
```

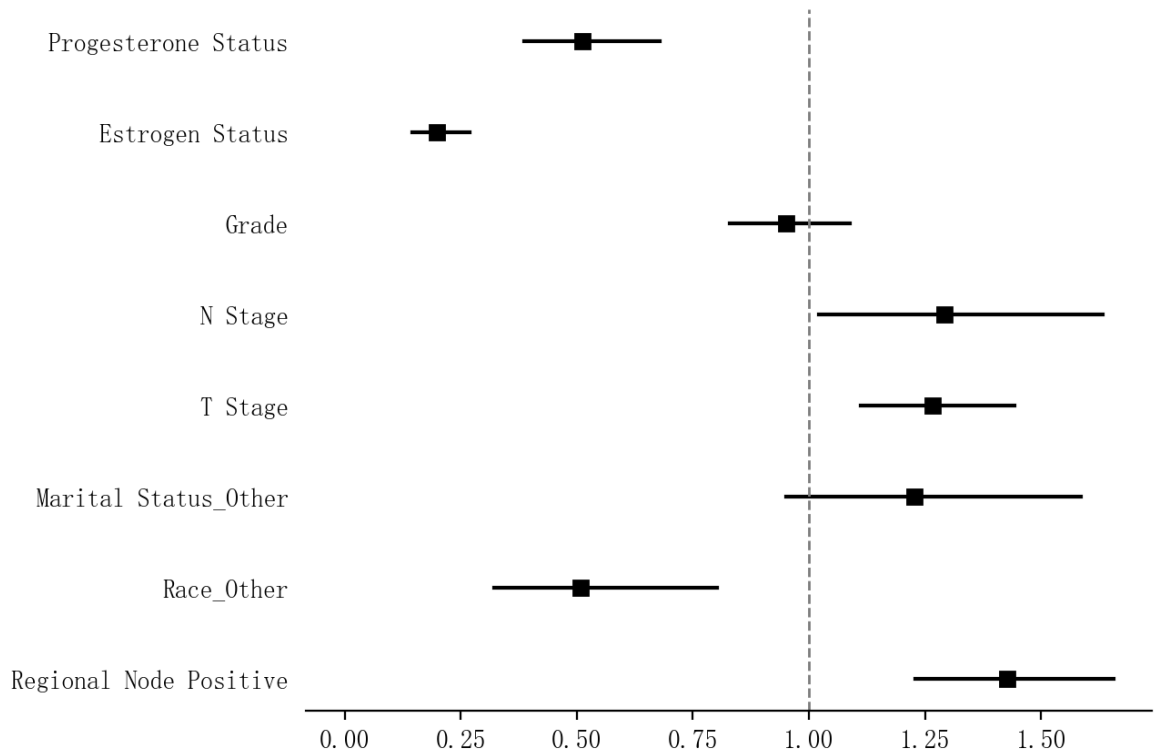
```

plt.plot([c[0],c[1]], [d,d], '-',0.25, color='gray',alpha=0.3)
plt.plot(c[2],d, 's',0.25, color='gray',alpha=0.3)
else:
    plt.plot([c[0],c[1]], [d,d], '-',0.25, color='black')
    plt.plot(c[2],d, 's',0.25, color='black')

plt.axvline(1, linestyle='--',color='grey',linewidth=1)

ax.tick_params(bottom=True, labelbottom=True, top=False, labeltop=False, left=False)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)

```



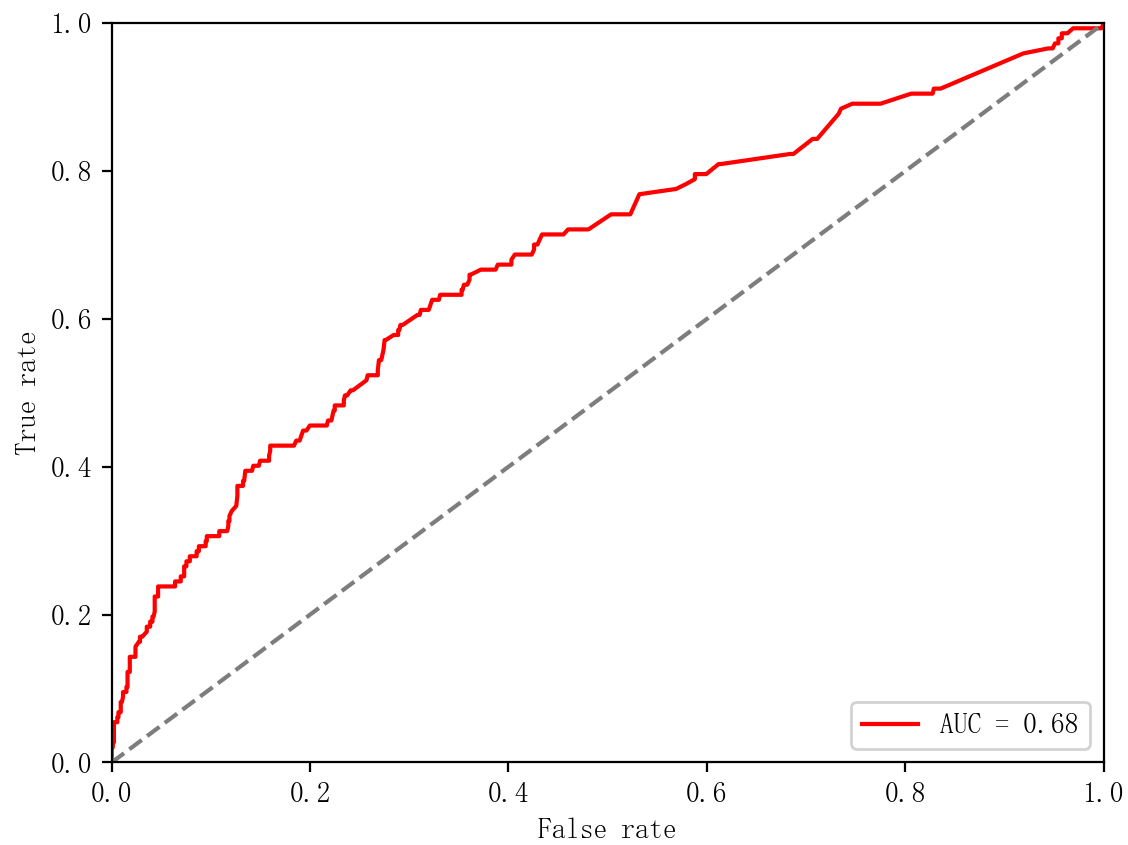
In [64]: `from sklearn.metrics import roc_curve, auc`

```

# 测试集分类预测
resu = model.predict(x_test[cols])
fpr, tpr, threshold = roc_curve(y_test, resu)

#计算ROC，并绘制曲线
rocauc = auc(fpr, tpr)
plt.plot(fpr, tpr, 'r', label='AUC = %0.2f' % rocauc)
plt.plot([0, 1], [0, 1], color='grey',linestyle='--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True rate')
plt.xlabel('False rate')
plt.legend(loc='lower right')
plt.show()

```



In []:

In []:

In []: